

UDP Socket Programming Assignment Report

Student Name: PARINITA DEVI

Roll Number: CS-BTC25-39

Timeout Used: 2.0 seconds(Configured via `socket.setdefaulttimeout(2.0)`)

Topic: Reliable UDP Client-Server Communication & Network Emulation

1. Introduction

The User Datagram Protocol (UDP) is a fast, connectionless transport protocol, but it does not guarantee packet delivery. The objective of this assignment is to design and implement a **Reliable UDP Application** using Python sockets. By introducing sequence numbers, Acknowledgments (*ACKs*), and a timeout mechanism, we ensure reliable delivery over an unpredictable network.

2. Mininet Topology

To test our application in a realistic environment, we used **Mininet** to emulate a virtual network.

- **Network Layout (net.png, nodes.png):** The setup consists of two host nodes (h1 and h2) connected via a single OpenFlow virtual switch (s1).
 - **Host h1:** Acts as the **Client** (10.0.0.1).
 - **Host h2:** Acts as the **Server** (10.0.0.2).
- **Connectivity Check (pingall.png):** We ran the `pingall` command to verify basic reachability. All host pairs achieved 0% packet loss, confirming a healthy network setup before launching our scripts.

3. Experimental Result Table

We tested our Reliable UDP setup under different packet loss conditions in Mininet to evaluate performance:

Experiment Scenario	Packet		ACKs Received	Retransmissions	Delivery	
	Loss Rate (%)	Packets Sent			Success Rate	Observation
Ideal Network	0%	10	10	0	100%	Fast transmission with minimum latency.
Low Loss Test	5%	10	10	1	100%	1 lost packet recovered via timeout rule.
Moderate Loss Test	10%	10	10	3	100%	Higher latency due to multiple retransmissions.

4. Short Answers to Review Questions

Q1: Why is ACK needed?

Standard UDP is a "fire-and-forget" protocol. An Acknowledgment (*ACK*) is sent back by the receiver to inform the sender that a specific packet arrived safely. Without *ACK*s, the sender has no way of knowing if data was delivered or lost.

Q2: Why is timeout needed?

If a packet or its corresponding *ACK* is lost in the network, the sender could wait forever. A timeout timer sets a maximum waiting limit. If no *ACK* arrives before the timer expires, the sender assumes the packet was lost and resends it automatically.

Q3: Why can duplicate messages occur?

Duplicates occur when a data packet reaches the receiver successfully, but the returning *ACK* is delayed or lost. Because the sender does not get the *ACK* in time, its timer expires and it resends the same message. The receiver uses sequence numbers to identify and drop duplicate packets.

Q4: What happens when packet loss increases?

When packet loss rises:

- More packets and *ACK*s are dropped, causing higher retransmission counts.
- The total transfer time increases because the sender frequently pauses to wait for timeouts.
- Overall network efficiency drops due to duplicate transmissions.

Q5: How is this method similar to TCP?

- **Guaranteed Delivery:** It uses Sequence Numbers and *ACK*s to ensure every packet reaches its destination.
- **Error Recovery:** It uses timeouts and automatic retransmissions to recover from network loss.

Q6: How is it different from TCP?

- **Implementation Layer:** Our method is built at the **Application Layer** on top of UDP, whereas TCP operates at the **Transport Layer** inside the Operating System kernel.
- **No Connection Overhead:** It does not require a 3-Way Handshake (*SYN*, *SYN – ACK*, *ACK*) before sending data.
- **Simplicity:** It lacks complex TCP features like sliding windows, flow control, and dynamic congestion control.

5. Conclusion

In this assignment, we successfully built a **Reliable Client-Server Application using UDP**. While standard UDP is unreliable by nature, our implementation proves that reliability can be achieved at the application level by introducing sequence numbers, timeouts, and *ACK*s. Testing via Mininet confirmed that even under lossy network conditions (5% and 10% loss), our protocol guarantees 100% eventual data delivery.